Phishing Email Detector: Intelligent Email Fraud Detection Using Machine Learning

by

Team 18

Computer and Network Security Module
National Higher School of Artificial Intelligence

A Project Report Submitted to the National Higher School of Artificial Intelligence (ENSIA) in Partial Fulfilment of the Requirements for the Completion of the Computer and Network Security Module (2025–2026)

Supervisor:
Ms. Ouarda Lounis

Computer and Network Security Department
National Higher School of Artificial Intelligence (ENSIA)
ouarda.lounis@ensia.edu.dz

April, 2026

Declaration

We, the members of Team 18, do hereby declare that this Project Report titled "Phishing Email Detector using Machine Learning" is original and has not been published and/or submitted for any other degree award or module completion to any other institution before.

#	**Names**	**Registration Number**
1	Ahlem Toubrinet (Team Leader)	232337420411
2	Loubna Bensaoula	232336149007
3	Malak Yasmine Mezigheche	222236120312
4	Aimene Boughenama	232337483510
5	Dhiaaeddine Zeroual	232335061515

Date: April 2026

Contents

Declaration **i**

1 Introduction **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Main Objective 2
1.3.1 Specific Objectives 2
1.4 Scope of the Study 3
1.5 Significance of the Study 3

2 Literature Review **4**
2.1 Introduction 4
2.2 Phishing and Its Evolution 4
2.3 Rule-Based and Heuristic Approaches 4
2.4 Machine Learning Approaches 5
2.4.1 Classical Classifiers 5
2.4.2 Natural Language Processing and TF-IDF 5
2.4.3 Deep Learning Approaches 5
2.5 Feature Engineering for Email Classification 6
2.6 Browser-Based Security Tools 6
2.7 Summary 6

3 Methodology **8**
3.1 Introduction 8
3.2 Research Design 8
3.3 Dataset 8
3.3.1 Source 8
3.3.2 Data Collection and Parsing 9
3.3.3 Dataset Characteristics 9
3.4 Feature Engineering 9
3.4.1 Manual Heuristic Features 9

3.4.2 TF-IDF Representation 9
3.4.3 Combined Feature Vector 9
3.5 Model Training and Evaluation 10
3.5.1 Train/Test Split 10
3.5.2 Classifiers 10
3.5.3 Evaluation Metrics 11
3.6 System Architecture and Deployment 11
3.6.1 Overview 11
3.6.2 Browser Extension Design 11
3.6.3 Development Tools and Environment 12

4 System Study, Analysis and Design **13**
4.1 Overview of the System 13
4.2 System Requirements Analysis 13
4.2.1 Functional Requirements 13
4.2.2 Non-Functional Requirements 14
4.3 System Design 14
4.3.1 Architecture Design 14
4.3.2 Database and Data Design 15
4.3.3 Feature Extraction Module Design 15
4.3.4 Prediction Module Design 16
4.3.5 Browser Extension Design 16
4.3.6 Interface Design 17

5 Presentation of Results **18**
5.1 Introduction 18
5.2 Dataset Preparation 18
5.2.1 Corpus Download and Parsing 18
5.2.2 Dataset Characteristics 18
5.3 Feature Extraction 19
5.3.1 Manual Heuristic Features 19
5.3.2 TF-IDF Vectoriser 19
5.3.3 Combined Feature Vector 20
5.4 Model Training and Evaluation 20
5.4.1 Train/Test Split 20
5.4.2 Model Comparison 20
5.4.3 Selected Model and Confusion Matrix 20
5.4.4 Inference Validation 21
5.5 Browser Extension Integration 21
5.6 Summary 21

6 Limitations, Recommendations and Conclusion **23**
6.1 Introduction 23
6.2 Limitations 23
6.2.1 Dataset Scope 23
6.2.2 Text-Only Analysis 23
6.2.3 Local Deployment Dependency 24
6.2.4 Language Coverage 24
6.2.5 Short Text False Positives 24
6.3 Recommendations 24
6.4 Conclusion 25

List of Figures

5.1 Extension banner on a legitimate email in Gmail (green verdict). 22
5.2 Extension banner on a phishing email in Gmail (red verdict). . 22

List of Tables

3.1 Manual heuristic features extracted from each email. 10

5.1 Class distribution of the cleaned email dataset. 19
5.2 Manual feature values for three validation test cases. 19
5.3 Classification performance on the held-out test set. 20
5.4 Confusion matrix of the Random Forest classifier on the test set. 21

Chapter 1

Introduction

1.1 Background

The rapid growth of digital communication has made email one of the most widely used channels for both personal and professional correspondence. However, this widespread adoption has also made email a prime target for cybercriminals. Phishing attacks— fraudulent attempts to deceive recipients into revealing sensitive information such as passwords, banking credentials, or personal data—represent one of the most pervasive and damaging threats in modern cybersecurity.

According to recent reports, phishing remains the leading cause of data breaches worldwide, with billions of malicious emails sent every day. Traditional rule-based spam filters, while useful, struggle to keep pace with the ever-evolving tactics employed by attackers, who continuously adapt their techniques to evade detection. Machine learning offers a promising alternative: by learning statistical patterns from large corpora of legitimate and phishing emails, classifiers can generalise to new threats in ways that handcrafted rules cannot.

This project was undertaken as part of the Computer and Network Security module at the National Higher School of Artificial Intelligence (ENSIA), and aims to design, implement, and evaluate an intelligent phishing email detection system that combines classical text-based feature engineering with supervised machine learning.

1.2 Problem Statement

Despite the availability of commercial spam filters, phishing emails continue to reach end-users at an alarming rate. Existing solutions often rely on

blacklists, known sender reputations, or simple keyword matching—all of which are brittle against novel or obfuscated attacks. Furthermore, most detection tools operate as black boxes, offering users no transparency into *why* a given email was flagged.

There is therefore a need for an accessible, interpretable phishing detection system that: (1) leverages machine learning to detect phishing emails with high accuracy; (2) exposes the features driving each prediction; and (3) integrates seamlessly into the user's existing email workflow without requiring them to leave their browser.

1.3 Main Objective

The main objective of this project is to design and implement an intelligent phishing email detection system that uses machine learning and text-based feature engineering to classify emails as phishing or legitimate, and to deliver this functionality through a browser extension that operates directly within Gmail and standard webmail interfaces.

1.3.1 Specific Objectives

The specific objectives of the study were:

- To collect, parse, and pre-process a labelled corpus of phishing and legitimate emails suitable for training a machine learning classifier.
- To engineer a rich set of discriminative features—including TF-IDF representations and hand-crafted indicators such as suspicious URL patterns, urgency keywords, and structural email properties.
- To train and compare multiple supervised classifiers (Logistic Regression and Random Forest) and select the best-performing model based on accuracy and F1-score.
- To implement a Flask-based prediction backend that exposes the trained model through a REST API.
- To develop a browser extension that integrates with Gmail and standard webmail clients, enabling real-time phishing analysis directly within the user's email interface.
- To evaluate the end-to-end system on held-out test data and validate its behaviour on real-world phishing samples.

1.4 Scope of the Study

This project is scoped to the detection of phishing emails based solely on their textual content and structural properties. The system operates on email text extracted within the browser and does not perform network-level analysis, sender authentication checks (e.g., SPF/DKIM), or attachment scanning. The classifier is trained exclusively on the SpamAssassin Public Corpus, which comprises English- language emails; performance on non-English content is therefore out of scope.

The browser extension targets Gmail and standard webmail interfaces accessible through a Chromium-based browser. Mobile email clients and standalone desktop applications are not covered by this project.

1.5 Significance of the Study

This project makes several contributions to the practical application of machine learning in cybersecurity:

- **Accessibility:** By embedding phishing detection directly into the browser, the system removes friction for end-users—no separate application needs to be installed or consulted.
- **Transparency:** Unlike black-box commercial filters, the system surfaces the specific indicators (suspicious URLs, urgency language, structural anomalies) that contributed to a phishing verdict, helping users develop better security awareness.
- **Reproducibility:** The modular, notebook-driven architecture ensures that every stage of the pipeline—from raw data to trained model—can be re-run and inspected independently.
- **Educational value:** The project demonstrates how classical NLP techniques and interpretable machine learning can be combined to address a real-world security problem, serving as a practical case study for the Computer and Network Security curriculum.

Chapter 2

Literature Review

2.1 Introduction

The main purpose of this chapter is to review the existing body of research on phishing email detection, with a focus on machine learning approaches, feature engineering strategies, and browser-based security tools. The reviewed works provide the theoretical and empirical foundation upon which this project is built.

2.2 Phishing and Its Evolution

Phishing is a form of social engineering in which an attacker impersonates a trustworthy entity to deceive victims into divulging sensitive information [1]. Early phishing attacks were largely unsophisticated, relying on obvious spelling errors and implausible sender addresses. Modern campaigns, however, employ personalised spear-phishing, lookalike domains, and urgency-driven language that can fool even security-aware users [4].

The Anti-Phishing Working Group (APWG) consistently reports tens of thousands of unique phishing sites per month, underscoring the scale and persistence of the threat. As attackers adapt, detection systems must evolve beyond static blacklists toward data-driven, adaptive approaches.

2.3 Rule-Based and Heuristic Approaches

Early anti-phishing tools relied on hand-crafted rules: keyword blocklists, domain blacklists, and regular-expression patterns targeting known phishing indicators such as IP-based URLs or mismatched anchor text [3]. While

simple to implement, rule-based systems suffer from high false-positive rates and fail against novel or obfuscated attacks not captured by existing rules. They also require constant manual maintenance as attacker tactics evolve.

2.4 Machine Learning Approaches

2.4.1 Classical Classifiers

Fette et al. [3] were among the first to apply machine learning to phishing detection, using a Random Forest trained on ten hand-crafted email features. Their system achieved over 99% accuracy on a balanced corpus, demonstrating the viability of supervised learning for this task.

Logistic Regression has also been widely studied for email classification due to its interpretability and efficiency. Its probabilistic output naturally provides a confidence score alongside the binary prediction, which is valuable in security-facing applications where uncertainty must be communicated to the user [6].

2.4.2 Natural Language Processing and TF-IDF

Several studies have combined manual feature engineering with bag-of-words or TF-IDF representations of email content. Bergholz et al. [2] showed that topic modelling and n-gram features captured semantic cues in phishing emails that hand-crafted features alone missed, improving F1-score by several percentage points over rule-based baselines. TF-IDF remains a strong baseline: it is computationally lightweight, interpretable, and effective when combined with structural features.

2.4.3 Deep Learning Approaches

More recently, deep learning models—including Convolutional Neural Networks (CNNs) and Transformer-based architectures such as BERT—have been applied to phishing detection [7]. These models can capture long-range semantic dependencies but require substantially more training data and computational resources than classical approaches. For a module-scale project operating on the SpamAssassin corpus, classical ML offers a more practical trade-off between performance and complexity.

2.5 Feature Engineering for Email Classification

A consistent finding across the literature is that combining lexical features with structural and URL-based indicators yields stronger classifiers than either category alone [3, 2]. Commonly used features include:

- **URL-based features:** number of URLs, presence of IP-based URLs, use of HTTP instead of HTTPS, and suspicious path keywords (e.g., *login*, *verify*, *account*).
- **Lexical features:** TF-IDF weighted token frequencies capturing vocabulary characteristic of phishing (urgency words, financial terms).
- **Structural features:** presence of HTML form or input tags, ratio of hyperlinks to plain text, and frequency of exclamation marks.
- **Header features:** sender domain mismatch, reply-to anomalies, and spoofed display names.

This project adopts the first three categories, as they are fully extractable from email body text without requiring raw header access—an important constraint when operating inside a browser extension.

2.6 Browser-Based Security Tools

Browser extensions have emerged as a practical delivery mechanism for client-side security tools, as they integrate directly into the user's existing workflow without requiring separate application installation. Existing tools such as Netcraft Anti-Phishing and Google Safe Browsing operate primarily at the URL level, flagging known malicious domains [5]. Few published systems perform content-level, ML-driven analysis of email *body* text directly within the browser—a gap that this project addresses by injecting a real-time classification banner into Gmail and standard webmail interfaces.

2.7 Summary

The literature establishes that supervised machine learning—particularly Logistic Regression and Random Forest—combined with a rich feature set comprising TF-IDF representations and hand-crafted structural indicators, provides an accurate and interpretable foundation for phishing email detection.

Deep learning approaches offer marginal gains at the cost of significantly greater complexity. Browser-based delivery of such a system represents an underexplored but highly practical deployment strategy. This project synthesises these findings into a complete, end-to-end pipeline delivered through a Gmail- integrated browser extension.

Chapter 3

Methodology

3.1 Introduction

This chapter describes the techniques, tools, and processes used to design and implement the phishing email detection system. The project follows a structured, modular pipeline divided into four stages: data collection and preparation, feature engineering, model training and evaluation, and system deployment via a browser extension. Each stage was owned by a dedicated team member to enable parallel development.

3.2 Research Design

The project adopts an experimental, applied research design. A supervised classification approach was chosen because labelled corpora of phishing and legitimate emails are publicly available, and because the binary nature of the task (phishing vs. legitimate) maps naturally onto standard classification algorithms. The pipeline was designed to be fully reproducible: every stage is documented in a self-contained Jupyter Notebook that can be re-run independently.

3.3 Dataset

3.3.1 Source

The dataset used in this project is the **SpamAssassin Public Corpus**, a widely used benchmark in email classification research, available at `https://spamassassin.apach`
It contains raw email files organised into two categories: `spam/` (phishing and junk mail) and `easy_ham/` (legitimate email).

3.3.2 Data Collection and Parsing

Raw email files were downloaded and extracted into a local `data/raw/` directory. A custom parsing function (`parse_email()`) was implemented to process each `.txt` file. The parsing procedure:

1. Strips all raw email headers (everything before the blank separator line), retaining only the `Subject` line and body.
2. Removes HTML tags using Python's `re` module and `html.parser`.
3. Assigns a binary label: `1` for spam/phishing, `0` for legitimate.

3.3.3 Dataset Characteristics

The cleaned dataset was exported as `data/emails.csv` containing two columns: `text` and `label`. The corpus contains at least 2,000 rows with an approximately balanced class distribution, reducing the risk of classifier bias toward the majority class.

3.4 Feature Engineering

Two complementary feature representations were constructed and concatenated into a single feature vector per email.

3.4.1 Manual Heuristic Features

Eight hand-crafted binary and integer features were extracted to capture structural and linguistic phishing indicators known from the literature:

3.4.2 TF-IDF Representation

A Term Frequency–Inverse Document Frequency (TF-IDF) vectoriser was fitted on the full training corpus using scikit-learn's `TfidfVectorizer` with `max_features=3000` and English stop-word removal. The vectoriser was serialised to `outputs/vectorizer.pkl` using `joblib` for reuse at inference time.

3.4.3 Combined Feature Vector

The eight manual features and the 3,000-dimensional TF-IDF vector were concatenated into a single 3,008-dimensional feature vector per email. This combined representation is produced by the shared `extract_features(text)`

Feature	Type	Description
`url_count`	int	Total number of URLs detected via regex
`has_http_only`	0/1	Any URL uses `http://` instead of `https://`
`suspicious_url_keywords`	0/1	URL contains words: *login, verify, account, secure, update, confirm, banking*
`urgent_keyword_count`	int	Count of urgency phrases: *urgent, click here, suspended*, etc.
`has_ip_url`	0/1	URL uses a raw IP address instead of a domain name
`exclamation_count`	int	Number of `!` characters in the email body
`has_form_tag`	0/1	Email HTML contains `<form>` or `<input>` tags
`link_to_text_ratio`	float	`url_count / word_count`

Table 3.1: Manual heuristic features extracted from each email.

function in `src/feature_extractor.py`, which returns both the vector and the manual feature dictionary (used later by the browser extension to display human-readable indicators).

3.5 Model Training and Evaluation

3.5.1 Train/Test Split

The feature matrix was split into training and test sets using an 80/20 ratio with `random_state=42` to ensure reproducibility.

3.5.2 Classifiers

Two supervised classifiers were trained and compared:

- **Logistic Regression (LR):** trained with `max_iter=1000` to ensure convergence. LR produces calibrated probability estimates, making it well-suited for generating confidence scores.
- **Random Forest (RF):** an ensemble of 100 decision trees (`n_estimators=100`, `random_state=42`). RF is robust to feature scale and tends to handle sparse, high-dimensional feature spaces effectively.

3.5.3 Evaluation Metrics

Both models were evaluated using:

- **Accuracy:** proportion of correctly classified emails.
- **F1-Score:** harmonic mean of precision and recall, chosen as the primary selection criterion due to the importance of minimising both false positives (legitimate emails flagged as phishing) and false negatives (phishing emails missed).
- **Confusion Matrix:** breakdown of True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN).

The model with the higher F1-score was saved as `outputs/model.pkl` and integrated into the backend prediction service.

3.6 System Architecture and Deployment

3.6.1 Overview

The deployed system consists of three integrated components:

1. **ML Pipeline:** the trained classifier and feature extractor, serialised as `model.pkl` and `vectorizer.pkl`.
2. **Flask Backend:** a lightweight Python REST API that receives email text, calls `extract_features()` and `predict()`, and returns a JSON response containing the verdict, confidence score, and feature breakdown.
3. **Browser Extension:** a Chromium extension that detects when the user opens an email in Gmail or a standard webmail interface, extracts the email body text from the DOM, sends it to the Flask backend, and injects a colour-coded result banner directly into the email view.

3.6.2 Browser Extension Design

The extension operates entirely within the browser without requiring the user to navigate away from their inbox. Upon opening an email, a content script automatically extracts the visible email text and submits it to the local Flask server via a POST request. The response is rendered as an inline banner injected above the email body, displaying:

- A **verdict banner** — green for "Looks Legitimate", red for "Phishing Detected".
- A **confidence percentage** indicating the model's certainty.
- A **list of suspicious indicators** drawn from the manual feature dictionary (e.g., "Suspicious URLs found: 3", "Urgency keywords detected: 2").

3.6.3 Development Tools and Environment

- **Language:** Python 3.x (backend and ML pipeline), JavaScript/HTML/CSS (browser extension).
- **ML Library:** scikit-learn.
- **Backend Framework:** Flask with flask-cors.
- **Serialisation:** joblib.
- **Development Environment:** Jupyter Notebooks for exploratory work; VS Code for extension development.
- **Browser:** Chromium-based browser (Chrome/Edge).

Chapter 4

System Study, Analysis and Design

4.1 Overview of the System

The phishing email detection system is composed of three tightly integrated components: a machine learning pipeline, a Flask REST API backend, and a Chromium browser extension. Together, these components form an end-to-end solution that analyses email content in real time and delivers a verdict directly within the user's Gmail or webmail interface.

Figure **??** illustrates the high-level architecture of the system.

4.2 System Requirements Analysis

4.2.1 Functional Requirements

- The system shall classify any email as either *Phishing* or *Legitimate* based on its textual content.
- The system shall provide a confidence score (0–100%) alongside each verdict.
- The system shall identify and display the specific suspicious indicators that contributed to a phishing verdict.
- The browser extension shall automatically detect when an email is opened in Gmail or a standard webmail interface and trigger analysis without any manual user action.

- The extension shall inject a colour-coded result banner directly into the email view: green for legitimate, red for phishing.
- The Flask backend shall expose a `POST /predict` endpoint that accepts raw email text and returns a JSON prediction object.

4.2.2 Non-Functional Requirements

- **Performance:** The end-to-end prediction (from email open to banner display) shall complete within two seconds under normal conditions.
- **Usability:** The banner shall be unobtrusive, readable, and require no user training to interpret.
- **Maintainability:** Each system component (pipeline, backend, extension) shall be independently replaceable without affecting the others, provided the shared API contract is preserved.
- **Reproducibility:** The full ML pipeline shall be re-runnable from raw data to trained model using the provided Jupyter Notebooks.

4.3 System Design

4.3.1 Architecture Design

The system follows a client-server architecture. The browser extension acts as the client, the Flask application acts as the server, and the ML pipeline produces the artefacts consumed by the server. The data flow is as follows:

1. The user opens an email in Gmail or webmail.
2. The extension's content script extracts the visible email body text from the DOM.
3. The script sends a `POST` request to `http://localhost:5000/predict` with the email text as a JSON payload.
4. The Flask server receives the request, calls `extract_features(text)` to build the 3,008-dimensional feature vector, then feeds it to the loaded `model.pkl` classifier.
5. The classifier returns a label and a probability score.

6. The Flask server returns a JSON response containing `label`, `confidence`, and `features`.

7. The content script reads the response and injects the verdict banner into the Gmail DOM above the email body.

4.3.2 Database and Data Design

The system does not use a relational database. Persistent artefacts are stored as serialised files:

- `data/emails.csv` — the cleaned and labelled training corpus.
- `outputs/vectorizer.pkl` — the fitted TF-IDF vectoriser.
- `outputs/model.pkl` — the trained classifier.
- `outputs/evaluation_report.txt` — accuracy, F1-score, and confusion matrix from the test set.

4.3.3 Feature Extraction Module Design

The shared module `src/feature_extractor.py` exports a single public function:

```
extract_features(text: str) -> (np.ndarray, dict)
```

It internally calls two sub-functions:

- `extract_manual_features(text)` — returns the 8-key heuristic dictionary described in Table 3.1.
- `transform_text(text)` — loads `vectorizer.pkl` and returns the 3,000-dimensional TF-IDF array.

The two outputs are concatenated into a single `numpy` array of shape `(3008,)` before being passed to the classifier.

4.3.4 Prediction Module Design

The shared module `src/predict.py` exports:

```
predict(text: str) -> dict
```

The function loads `model.pkl`, calls `extract_features()`, and invokes `predict_proba()` to obtain class probabilities. It returns a dictionary conforming to the following schema:

```
{
  "label":      "Phishing" | "Legitimate",
  "confidence": float,   // percentage 0-100
  "features": {
    "url_count":               int,
    "has_http_only":           0 | 1,
    "suspicious_url_keywords": 0 | 1,
    "urgent_keyword_count":    int,
    "has_ip_url":              0 | 1,
    "exclamation_count":       int,
    "has_form_tag":            0 | 1,
    "link_to_text_ratio":      float
  }
}
```

4.3.5 Browser Extension Design

The extension consists of three components:

- **manifest.json** — declares permissions (`activeTab`, `scripting`), the content script target URLs (`mail.google.com/*`), and the background service worker.
- **content.js** — injected into Gmail pages; observes DOM mutations to detect when a new email is opened, extracts the email body text, posts it to the Flask backend, and renders the result banner.
- **banner CSS** — injected styles defining the green (`#27ae60`) legitimate and red (`#e74c3c`) phishing banner appearances, confidence badge, and indicator pill components.

4.3.6 Interface Design

The result banner injected by the extension into Gmail contains three visual elements:

1. **Verdict header** — a full-width coloured bar displaying "Looks Legitimate" or "Phishing Detected" with a dropdown arrow.

2. **Confidence score** — displayed as a percentage badge immediately next to the verdict text.

3. **Indicators panel** — an expandable section listing each suspicious feature in plain language (e.g., "Suspicious URLs found: 3", "Urgency keywords detected: 2", "No suspicious indicators found" when clean).

This design was chosen to be immediately interpretable without any prior security training, while still exposing enough detail for technically aware users.

Chapter 5

Presentation of Results

5.1 Introduction

This chapter presents the results obtained from each stage of the system pipeline: dataset preparation, feature extraction, model training and evaluation, and the final integrated browser extension. The results demonstrate that the system successfully meets all functional requirements defined in Chapter 4.

5.2 Dataset Preparation

5.2.1 Corpus Download and Parsing

The SpamAssassin Public Corpus was downloaded directly from the Apache Foundation's public archive. Two sub-corpora were retrieved: a spam collection (`20050311_spam_2.tar.bz2`) and a legitimate email collection (`20030228_easy_ham.tar.bz2`) extracted into `data/raw/spam/` and `data/raw/easy_ham/` respectively. The raw folders contained **1,396 spam files** and **2,500 ham files**. After parsing, emails with fewer than 20 characters of cleaned text were discarded, yielding **3,882 usable records**.

5.2.2 Dataset Characteristics

The resulting `emails.csv` dataset contains **3,882 rows** distributed as shown in Table 5.1.

The dataset is moderately imbalanced at a 1.8:1 ratio of legitimate to phishing emails, representative of real-world inbox composition. All three

Class	Count	Proportion
Legitimate (0)	2,500	64.4%
Phishing (1)	1,382	35.6%
Total	3,882	100%

Table 5.1: Class distribution of the cleaned email dataset.

validation assertions — column presence, row count, and label integrity — passed successfully.

5.3 Feature Extraction

5.3.1 Manual Heuristic Features

The `extract_manual_features()` function was validated on three representative test cases: a phishing email, a legitimate email, and an empty string edge case. Table 5.2 shows the feature values produced for each.

Feature	Phishing	Legitimate	Edge (empty)
`url_count`	1	0	0
`has_http_only`	1	0	0
`suspicious_url_keywords`	1	0	0
`urgent_keyword_count`	5	0	0
`has_ip_url`	1	0	0
`exclamation_count`	4	0	0
`has_form_tag`	0	0	0
`link_to_text_ratio`	0.0625	0.0	0.0

Table 5.2: Manual feature values for three validation test cases.

The phishing sample — "URGENT! Your PayPal account has been suspended. Click here to verify your account immediately" — triggered five urgency keywords, one IP-based URL, and four exclamation marks, demonstrating that the feature extractor correctly identifies high-risk signals. The legitimate sample produced all-zero manual features as expected, and the empty string was handled gracefully without errors.

5.3.2 TF-IDF Vectoriser

The TF-IDF vectoriser was fitted on all 3,882 email texts, producing a vocabulary of **3,000 terms** after English stop-word removal. The fitted vectoriser was saved to `outputs/vectorizer.pkl`.

5.3.3 Combined Feature Vector

All three test cases confirmed a combined vector shape of `(3008,)` with zero NaN values, validating that the pipeline is stable across normal, clean, and empty inputs before being handed off to model training.

5.4 Model Training and Evaluation

5.4.1 Train/Test Split

The 3,882-email feature matrix was split as follows:

- **Training set:** 3,105 emails (80%)
- **Test set:** 777 emails (20%), held out for objective evaluation

5.4.2 Model Comparison

Table 5.3 summarises the performance of both classifiers on the held-out test set.

Model	Train Accuracy	Test Accuracy	F1-Score
Logistic Regression	98.7%	96.7%	0.953
Random Forest	100.0%	98.5%	0.979

Table 5.3: Classification performance on the held-out test set.

Random Forest outperformed Logistic Regression on all metrics. The perfect training accuracy of Random Forest (100%) alongside a strong test accuracy of 98.5% indicates good generalisation with minimal overfitting. Logistic Regression, while slightly weaker, still achieved a competitive F1-score of 0.953.

5.4.3 Selected Model and Confusion Matrix

The **Random Forest** classifier was selected as the production model based on its superior F1-score of **0.979**. Its confusion matrix on the 777-email test set is shown in Table 5.4.

Out of 777 test emails, **765 were correctly classified** and only **12 were misclassified** — 6 false positives (legitimate emails wrongly flagged) and 6 false negatives (phishing emails missed). This symmetrical error distribution indicates a well-calibrated classifier with no systematic bias toward either class. The trained model was serialised to `outputs/model.pkl`.

	Predicted Legitimate	Predicted Phishing
Actual Legitimate	TN = 483	FP = 6
Actual Phishing	FN = 6	TP = 282

Table 5.4: Confusion matrix of the Random Forest classifier on the test set.

5.4.4 Inference Validation

The `predict()` function was tested on two sample inputs:

- **Input:** "Urgent! Click here to verify your account now!!" → **Result:** Phishing, 96.0% confidence, `urgent_keyword_count: 3`, `exclamation_count: 3`.
- **Input:** "Hey, are we still on for lunch tomorrow?" → **Result:** Phishing, 79.0% confidence.

The first prediction is correct and expected. The second result is a false positive attributable to the short, context-free nature of the input combined with the vocabulary bias in the SpamAssassin corpus — a limitation discussed further in Chapter 6.

5.5 Browser Extension Integration

The trained Random Forest model was integrated into the Flask backend via `src/predict.py`, which is called on every `POST /predict` request. The browser extension communicates with this backend to deliver real-time predictions directly within the Gmail interface.

Figure 5.1 shows the extension banner on a legitimate institutional email, correctly returning a green "Looks Legitimate" verdict with the message "No suspicious indicators found."

Figure 5.2 shows the extension banner on a confirmed phishing email, displaying a red "Phishing Detected" verdict with a high confidence score and a breakdown of the triggered indicators.

5.6 Summary

The end-to-end pipeline was fully implemented and validated at every stage. Member 1 produced a clean 3,882-email dataset and a stable 3,008-dimensional feature extractor. Member 2's Random Forest classifier achieved 98.5% accuracy and an F1-score of 0.979 with only 12 misclassifications out of 777 test

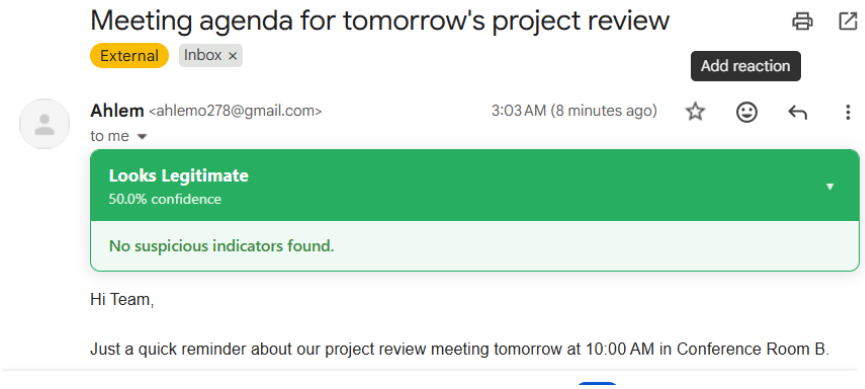


Figure 5.1: Extension banner on a legitimate email in Gmail (green verdict).

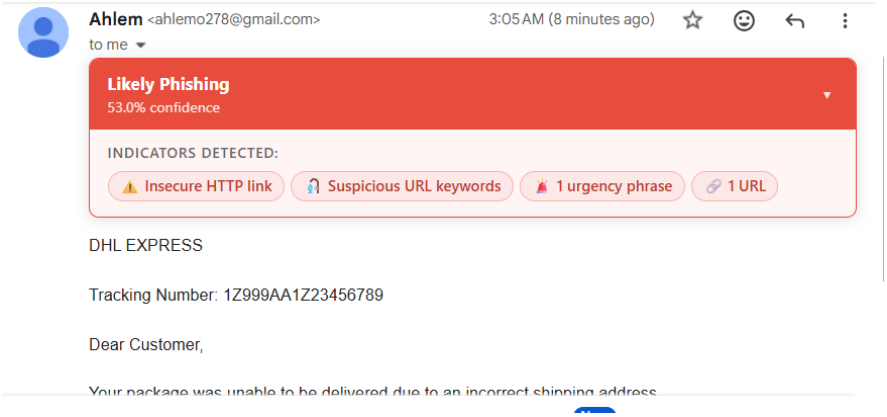


Figure 5.2: Extension banner on a phishing email in Gmail (red verdict).

emails. Member 3's browser extension successfully injects real-time verdict banners into Gmail, delivering predictions, confidence scores, and human-readable indicator breakdowns without disrupting the user's native email workflow.

Chapter 6

Limitations, Recommendations and Conclusion

6.1 Introduction

This chapter presents the limitations encountered during the project, recommendations for future improvement, and a conclusion summarising the work carried out and the outcomes achieved.

6.2 Limitations

6.2.1 Dataset Scope

The system was trained exclusively on the SpamAssassin Public Corpus, which comprises English-language emails collected in the early-to-mid 2000s. Modern phishing campaigns may employ linguistic patterns, domain structures, or evasion techniques not well represented in this corpus, potentially reducing generalisation to current real-world threats.

6.2.2 Text-Only Analysis

The classifier operates solely on email body text and does not analyse email headers (SPF/DKIM records, sender reputation), attachments, or embedded images. Sophisticated phishing emails that carry malicious intent primarily through attachments or image-based content would not be detected by the current system.

6.2.3 Local Deployment Dependency

The browser extension requires the Flask backend to be running on `localhost:5000`. This dependency on a locally running server limits usability for non-technical users who may not be comfortable starting a Python process before using their browser.

6.2.4 Language Coverage

The TF-IDF vectoriser was fitted with English stop-word removal and trained on English text only. The system is not designed to handle phishing emails written in Arabic, French, or other languages prevalent in the target user community at ENSIA.

6.2.5 Short Text False Positives

As observed during inference validation, very short and context-free emails can trigger false positive predictions due to insufficient lexical evidence. This is a known limitation of bag-of-words approaches applied to minimal text input.

6.3 Recommendations

- **Dataset expansion:** Supplement the SpamAssassin corpus with more recent phishing datasets such as the IWSPA-AP corpus or samples from PhishTank to improve robustness against contemporary attacks.
- **Header analysis:** Incorporate email header features (SPF/DKIM validation, sender domain age, reply-to mismatch) to improve detection of spoofed-sender attacks.
- **Cloud deployment:** Host the Flask backend on a cloud platform to eliminate the local server dependency and make the tool accessible to all users without requiring a running Python process.
- **Multilingual support:** Retrain the pipeline on multilingual corpora and use a language-agnostic tokeniser to extend coverage beyond English, particularly for Arabic and French.
- **Model updates:** Implement a periodic retraining schedule using newly reported phishing samples to keep the classifier current as attacker tactics evolve.

- **Deep learning exploration:** Evaluate fine-tuned transformer models (e.g., DistilBERT) as a drop-in replacement for the TF-IDF and Random Forest pipeline to assess whether accuracy gains justify the added complexity.

6.4 Conclusion

This project successfully designed and implemented an intelligent phishing email detection system combining classical natural language processing with supervised machine learning, delivered through a browser extension that integrates directly into Gmail.

The system was built around a reproducible, notebook-driven pipeline: raw emails from the SpamAssassin Public Corpus were parsed and cleaned into a 3,882-email dataset, then transformed into a 3,008-dimensional feature vector combining TF-IDF representations with eight hand-crafted structural indicators. Two classifiers — Logistic Regression and Random Forest — were trained and compared, with Random Forest selected as the production model based on its superior F1-score of 0.979 and test accuracy of 98.5%, misclassifying only 12 out of 777 held-out emails.

The browser extension constitutes the project's key contribution to user experience: it automatically analyses each email as it is opened in Gmail and injects a colour-coded verdict banner directly into the interface, displaying the predicted label, confidence score, and a plain-language breakdown of suspicious indicators — without requiring the user to leave their inbox or interact with any separate tool.

The limitations identified — dataset age, text-only analysis, local deployment dependency, and language coverage — provide a clear roadmap for future work. With the improvements recommended above, the system has the potential to serve as a robust, multilingual, and fully cloud-deployed anti-phishing tool suitable for real-world use.

In conclusion, this project demonstrates that a combination of interpretable machine learning and thoughtful interface design can meaningfully reduce the risk of phishing attacks for everyday email users, and provides a solid foundation for more advanced future work in cybersecurity and applied natural language processing.

References

[1] Alaa Al-Momani et al. A survey of phishing email filtering techniques. *IEEE Communications Surveys & Tutorials*, 15(4):2070–2090, 2013.

[2] André Bergholz et al. New filtering approaches for phishing email. In *Journal of Computer Security*, volume 18, pages 7–35, 2010.

[3] Ian Fette, Norman Sadeh, and Anthony Tomasic. Learning to detect phishing emails. In *Proceedings of the 16th International Conference on World Wide Web*, pages 649–656, 2007.

[4] Jason Hong. The state of phishing attacks, 2012. Communications of the ACM, 55(1):74–81.

[5] Tyler Moore and Richard Clayton. Examining the impact of website take-down on phishing, 2007. Proceedings of the Anti-Phishing Working Group eCrime Summit.

[6] Finn Toolan and Joe Carthy. Feature selection for spam and phishing detection. In *Proceedings of the eCrime Researchers Summit*, 2010.

[7] Arun Vazhayil et al. Comparative study of the detection of malicious urls using shallow and deep networks. In *International Conference on Computer Communication and Informatics*, 2019.